Atividade de registro semana 26 de versionamento
Nome: Arthur Saraiva de França 3b.

Parte 1:

Etapa 1:

-O que pode acontecer se um código com testes apresentando falhas entrar na main?

Pode quebrar funcionalidades já existentes no sistema, introduzir bugs em produção, interromper o trabalho de outros desenvolvedores e gerar instabilidade para os clientes finais.

-Por que depender apenas da revisão humana pode gerar erro?

Porque as pessoas podem ficar cansadas, deixar passar falhas por falta de atenção, esquecer de rodar os testes localmente ou não perceber problemas sutis de formatação e sintaxe.

Etapa 2:

-O que os checks automatizados verificam antes da revisão?

Verificam de forma autônoma se o código compila perfeitamente, se todos os testes unitários continuam passando, se a formatação segue o padrão do projeto e se não existem vulnerabilidades conhecidas.

-Dê dois exemplos de problemas que eles podem detectar.

Erros de sintaxe ou compilação (build) e testes unitários quebrados que falharam após as novas alterações.

Etapa 3:

-Por que bloquear o merge é melhor do que apenas avisar?

Porque o bloqueio atua como uma barreira física intransponível no sistema, garantindo que nenhum código problemático entre na main por engano, pressa ou descuido de algum desenvolvedor.

-O que pode acontecer se alguém ignorar um aviso?

O código instável será integrado à branch principal, quebrando a esteira de deploy e gerando retrabalho para toda a equipe consertar o erro.

-Como essa regra muda o comportamento do time?

A regra cria uma cultura de maior responsabilidade individual. Os desenvolvedores passam a testar e revisar melhor seu próprio código antes de abrir uma PR, sabendo que o sistema não permitirá o merge se houver falhas.

Etapa 4:

-Por que definir um limite mínimo de cobertura?

Para garantir que a equipe mantenha o hábito de criar testes para as novas funcionalidades, evitando que o projeto cresça sem uma rede de proteção de código testado.

-Cobertura alta garante qualidade total? Justifique.

Não, pois a cobertura apenas indica que as linhas de código foram executadas durante os testes, mas não garante que os cenários de teste escritos são inteligentes, cobrem cenários reais de erro ou validam as regras de negócio corretamente.

-Mesmo assim, por que a cobertura ainda é útil?

Ela é útil como uma métrica de alerta. Uma queda brusca na porcentagem mostra claramente que um código novo e complexo foi inserido no sistema sem nenhuma proteção ou validação.

Etapa 5:

-Essa PR deve ser aprovada? Por quê?

Não. Ela viola todas as regras de qualidade estabelecidas, contendo testes ativamente quebrados, falta de testes novos e ausência de validação humana.

-O que precisa ser corrigido antes do merge?

O autor precisa consertar os testes que estão falhando, escrever novos testes para elevar a cobertura de volta ao limite mínimo aceitável e solicitar a revisão e aprovação de outro membro do time.

Etapa 6:

Checklist para aprovação de PR:

-Todos os testes automatizados passaram com sucesso?

-A cobertura de código está dentro do limite mínimo estabelecido?

-A revisão humana foi concluída e aprovada pelos revisores?

-Como esse checklist ajuda a evitar problemas na main?

Ele funciona como um filtro rigoroso de qualidade. Ao exigir que todas as condições sejam atendidas simultaneamente, o time cria uma blindagem em torno da main, assegurando que o código integrado esteja sempre limpo, testado e validado.

Parte 2:

Etapa 1:

-O que está faltando nessa revisão?

Falta clareza, feedbacks construtivos, apontamentos técnicos específicos e profissionalismo por parte do revisor.

-O problema está só no código?

Não, o problema também está na comunicação agressiva, na falta de empatia e na ausência de justificativas e explicações sobre o que precisa ser corrigido.

-Como isso pode gerar conflito no time?

Gera um ambiente de trabalho tóxico, desmotiva os desenvolvedores, cria barreiras na comunicação e destrói a confiança entre os membros da equipe.

Etapa 2:

-Por que isso ajuda quem revisa?

Ajuda porque o revisor ganha contexto imediato sobre as alterações, entendendo as motivações do autor e sabendo exatamente como testar os cenários modificados sem perder tempo investigando o código do zero.

-O que acontece quando essas informações não existem?

O processo de revisão fica muito mais lento, há maior chance de falhas passarem despercebidas e o revisor precisa ficar adivinhando o propósito daquela alteração.

Etapa 3:

-Quais são sugestões (LGTM)?

Os itens 1 (trocar nome de variável) e 3 (reorganizar comentário).

-Quais são problemas reais (change-request)?

Os itens 2 (quebra do código com valor vazio) e 4 (falta de testes da funcionalidade principal).

-O que diferencia uma sugestão de um bloqueio?

A sugestão aborda melhorias estéticas, de estilo ou legibilidade que não impedem o sistema de funcionar. O bloqueio indica uma falha técnica real, um bug em potencial ou a falta de requisitos obrigatórios que tornam o código inseguro para ir para produção.

-Quando usar LGTM?

Quando o código cumpre os requisitos propostos, funciona corretamente nos testes e possui apenas pontos de melhorias opcionais que não afetam a segurança ou a estabilidade da aplicação.

-Quando pedir um change-request?

Quando forem identificados erros de lógica, bugs, ausência de testes essenciais, falhas de segurança ou violações graves dos padrões do projeto.

Etapa 4:

-Qual comentário ajuda mais?

O comentário B.

-Qual comentário tende a gerar menos atrito?

O comentário B.

-O que um bom comentário precisa ser/ter?

Precisa ser específico, objetivo, focado no código e não na pessoa, além de explicar o motivo do problema e propor uma alternativa ou solução viável.

Regras para bons comentários:

1. Explique a razão técnica por trás da alteração solicitada.
2. Seja gentil, empático e evite termos agressivos ou julgamentos pessoais.
3. Ofereça um exemplo claro ou uma sugestão de como o trecho pode ser melhorado.